



Procesamiento Digital de Señales

Ingeniería en Computación

Laboratorio 1

Muestreo con MCXN947

FRDM-MCXN947 · MCUXpresso IDE · SDK MCUXpresso

Integrante	Matrícula	Correo
Costamagna, Matias Javier	44.512.437	matias.costamagna@mi.unc.edu.ar
Moreyra, Julián	45.214.537	julian.moreyra@mi.unc.edu.ar
Rivera, Mariano	45.432.921	luismarianorivera.25@mi.unc.edu.ar
Rojas, Facundo Nicolás	44.629.222	facundo.rojas.222@mi.unc.edu.ar

Docentes: Rossi, Roberto Raúl · Parlanti, Gustavo Fernando · Molina, Germán Alcides

Fecha de entrega: 2 de octubre de 2026

Índice

1. Introducción	2
1.1. Requerimientos	2
1.2. Correspondencia con la presentación	2
2. Configuración del ambiente	3
2.1. Herramientas utilizadas	3
2.2. Creación del proyecto	3
2.3. Relojes	3
2.4. Pines	4
2.5. Periféricos	4
3. Diseño de la aplicación	9
3.1. Arquitectura general	9
3.2. Generación de las velocidades de muestreo	9
3.3. Máquina de estados	10
3.4. Formato q15 y conversiones	10
3.5. Rutinas de interrupción	11
4. Implementación	12
4.1. Constantes y variables globales	12
4.2. Programa principal	12
4.3. Cambio de velocidad	12
4.4. Rutinas de interrupción	12
5. Verificación	13
5.1. Procedimiento	13
5.2. Frecuencia de muestreo medida	13
5.3. Señal de entrada y salida	14
5.4. Cumplimiento de los requerimientos	15
6. Conclusiones	16
A. Código fuente	17

1 Introducción

Laboratorio 1 — Muestreo con MCXN947

Objetivo. Manejar el módulo del conversor A/D integrado en el MCU MCXN947.

Descripción. Realizar un programa aplicativo que sea capaz de digitalizar una señal a través del módulo del ADC disponible en la placa MCXN947 a distintas velocidades de muestreo; las velocidades requeridas son 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s. Los cambios de las velocidades de muestreo serán realizados con una de las teclas de la placa de evaluación, en forma de un buffer circular. Cada velocidad de muestreo se indicará a través de un color RGB del LED. Con otra tecla de la placa se habilitará la adquisición o se parará la misma (Run/Stop). Los valores adquiridos serán almacenados en memoria en un buffer circular de 512 muestras del tipo q15 (fraccional de 15 bits) y a su vez serán enviados a través del DAC (de 12 bits).

Presentación.

1. Mostrar el programa corriendo.
2. Subir un archivo conteniendo lo siguiente:
 - Configuración del ambiente (pantallas con timers, ADC, etc.).
 - Diagrama de diseño de la aplicación explicando sus partes.
 - Código fuente de la misma con todo el proyecto.

1.1 Requerimientos

De la consigna se desprenden los siguientes requerimientos, que se usan como guía para el diseño y como lista de verificación en la sección 5:

R1 Muestrear una señal analógica con el ADC a 8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s.

R2 Recorrer las velocidades en forma circular con una tecla de la placa.

R3 Indicar la velocidad activa con un color del LED RGB.

R4 Habilitar o detener la adquisición con otra tecla (Run/Stop).

R5 Enviar cada muestra al DAC de 12 bits.

1.2 Correspondencia con la presentación

Tabla 1: Dónde se encuentra cada ítem pedido en la presentación.

Ítem pedido	Ubicación en este informe
Configuración del ambiente	Sección 2
Diagrama de diseño de la aplicación	Sección 3
Código fuente del proyecto	Sección 4 y Anexo A
Programa corriendo	Demostración en clase; evidencia en sección 5

2 Configuración del ambiente

2.1 Herramientas utilizadas

Tabla 2: Hardware y software del entorno de trabajo.

Elemento	Versión / modelo
Placa de evaluación	FRDM-MCXN947 (Cortex-M33, núcleo <code>cm33_core0</code>)
IDE	MCUXpresso IDE v25.6.136
SDK	SDK_26.06.00_FRDM-MCXN947
Herramientas de config.	MCUXpresso Config Tools (Clocks, Pins, Peripherals)
Proyecto base	Ejemplo <code>lpadc_interrupt_cm33_core0</code> del SDK
Bibliotecas	Drivers del SDK y CMSIS-DSP (tipo <code>q15_t</code>)
Generador de señales	
Osciloscopio	

2.2 Creación del proyecto

Para crear el proyecto TP1 se importó del SDK de la FRDM-MCXN947 el ejemplo `lpadc_interrupt`, que ya trae la inicialización de la placa y el driver del LPADC. Sobre esa base:

1. Se instaló el SDK de la FRDM-MCXN947 en el IDE y se importó el ejemplo para el núcleo `cm33_core0`.
2. Se agregaron los drivers `ctimer`, `dac` (LPDAC), `inputmux`, `gpio`, `vref` y `spc`, y la biblioteca CMSIS-DSP para disponer del tipo `q15_t` (`arm_math.h`).
3. Con Config Tools se configuraron los relojes, los pines y los periféricos; el código generado quedó en `board/clock_config.c`, `board/pin_mux.c` y `board/peripherals.c`.
4. La lógica de la aplicación se escribió en `source/TP1.c`.
5. La depuración se hace con el MCU-Link integrado de la placa, con la configuración guardada en TP1 `LinkServer Debug.launch`.

2.3 Relojes

El reloj del núcleo determina cuántos ciclos hay disponibles para atender cada muestra, y el reloj de entrada del timer fija la resolución con la que se pueden generar las velocidades de muestreo. Se usa la configuración `BOARD_BootClockPLL150M`: el PLL0 toma el FRO de 48 MHz y lo lleva a

$$f_{\text{PLL0}} = \frac{48 \text{ MHz}}{N_{\text{DIV}}} \cdot \frac{M_{\text{DIV}}}{2} = \frac{48 \text{ MHz}}{8} \cdot \frac{50}{2} = 150 \text{ MHz}. \quad (1)$$

Con esa frecuencia se alimentan el núcleo y el CTIMER0. El ADC0 y el DAC0 toman el `FRO_HF` de 48 MHz, suficiente para ambos conversores.

Tabla 3: Frecuencias de reloj de los módulos utilizados.

Módulo	Fuente	Divisor	Frecuencia
Núcleo (<code>main_clk</code>)	PLL0	1	150 MHz
CTIMER0	PLL0	1	150 MHz
LPADC0	FRO_HF	1	48 MHz
DAC0	FRO_HF	1	48 MHz

2.4 Pines

Tabla 4: Asignación de pines y su ubicación en los conectores de la placa.

Función	Pin	Periférico	Conector / observación
Entrada analógica	ADC0_A0	LPADC0 canal A0	J4[2] (Arduino A0), 0 V a 3,3 V
Salida analógica	P4_2	DAC0_OUT	J1[4]
Disparo del ADC	P0_19	CT0_MAT3	J2[13], punto de prueba
LED rojo	P0_10	GPIO0	Activo en bajo
LED verde	P0_27	GPIO0	Activo en bajo
LED azul	P1_2	GPIO1	Activo en bajo
Tecla SW2 (Run/Stop)	P0_23	GPIO0, canal IRQ 1	Flanco descendente
Tecla SW3 (velocidad)	P0_6	GPIO0, canal IRQ 0	Flanco descendente, pull-up interno
Consola de depuración	P1_8, P1_9	LP_FLEXCOMM4	MCU-Link

La salida del Match 3 del CTIMER0 se saca además por P0_19: esa señal es la que dispara al ADC, así que con el osciloscopio en J2[13] se mide directamente la frecuencia de muestreo (sección 5).

#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction	GPIO initial state	GPIO interrupt selection	GPIO interrupt
C14	GPIO0	GPIO, 6	<-	[C14] PIO0_6	P0_6/J23[7]/SW3[3]/SW3[4]	SW3	Input	n/a	Output 0	Interrupt on falling edge
B7	GPIO0	GPIO, 23	<-	[B7] PIO0_23	P0_23/SJ9[1]/SW2[3]/SW2[4]	SW2	Input	n/a	Output 1	Interrupt on falling edge

Figura 1: Config Tools — Pins. Ruteo de pines: Teclas.

#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction
A1	LP_FLEXCOMM4	P0, SDO/RXD/SDA	<->	[A1] FC4_P0	P1_8/J9[32]	DEBUG_UART_RX	Input/Output
B1	LP_FLEXCOMM4	P1, SCK/TXD/SCL	<->	[B1] FC4_P1	P1_9/J9[30]	DEBUG_UART_TX	Input/Output
T1	DAC0	OUT	->	[T1] DAC0_OUT	P4_2/J1[4]	n/a	Output
n/a	ADC0	TRG_CH_0	<-	CTIMER0_MATCH3	n/a	n/a	Input
C9	CTIMER0	MATCH, 3	->	[C9] CT0_MAT3	P0_19/J2[13]/J7[1]	n/a	Output
P3	ADC0	A, 0	<-	[P3] ADC0_A0	ADC0_A0/ARD_A0/J4[2]	n/a	Input

Figura 2: Config Tools — Pins. Ruteo de pines: ADC0_A0, DAC0_OUT y CT0_MAT.

#	Peripheral	Signal	Arrow	Routed pin/signal	Label	Identifier	Direction	GPIO initial state
C4	GPIO1	GPIO, 2	->	[C4] PIO1_2	P1_2/SJ4[1]	LED_BLUE	Output	Logical 0
B12	GPIO0	GPIO, 10	->	[B12] PIO0_10	P0_10/SJ5[1]	LED_RED	Output	Logical 0
E10	GPIO0	GPIO, 27	->	[E10] PIO0_27	P0_27/SJ6[1]	LED_GREEN	Output	Logical 0

Figura 3: Config Tools — Pins. Ruteo de pines: LED RGB.

2.5 Periféricos

2.5.1 Timer de muestreo (CTIMER0)

El CTIMER0 cuenta a 150 MHz (prescaler 0) y usa el canal de *match* 3 con reinicio del contador y salida en modo *toggle*. Cada vez que la cuenta llega a MR la salida cambia de estado, así que en P0_19 aparece una onda cuadrada de período $2(MR+1)$ ciclos del timer. Esa señal se conecta por el **INPUTMUX** a la entrada de disparo 0 del LPADC0 (`kINPUTMUX_Ctimer0M3ToAdc0Trigger`), que arranca una conversión por período. De esta forma el instante de muestreo lo fija el hardware y no depende de la latencia de las interrupciones: el CTIMER no genera interrupciones.

Tabla 5: Configuración del CTIMER0.

Parámetro	Valor
Modo	Timer (<code>kCTIMER_TimerMode</code>)
Reloj / prescaler	150 MHz / 0
Canal	Match 3
Acción en el match	Reinicio del contador, sin detener, sin interrupción
Salida	Toggle (<code>kCTIMER_Output_Toggle</code>), P0_19
Valor inicial de MR	9374 (generado por Config Tools)
Destino del disparo	INPUTMUX → LPADC0, trigger 0

Figura 4: Config Tools — CTIMER0. Modo timer, prescaler, canal de match 3 con reinicio y salida en toggle.

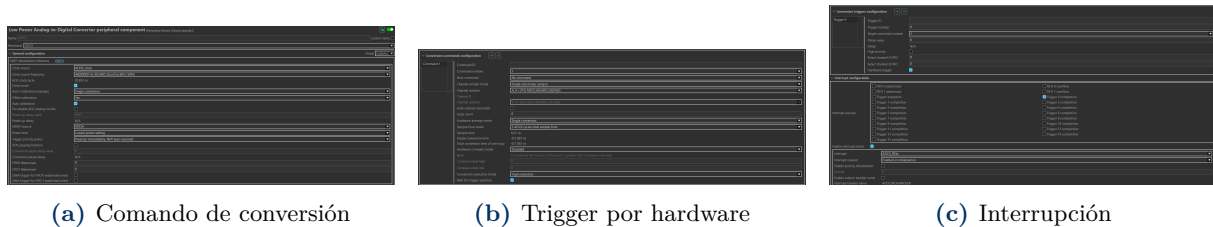
2.5.2 Conversor A/D (LPADC0)

El ADC0 se usa en resolución estándar (12 bit, single-ended) sobre el canal A0. El trigger 0, habilitado por hardware, ejecuta el comando 1, que convierte una vez y deja el resultado en la FIFO0. Al completarse el trigger se genera la interrupción `ADC0_IRQn`. Antes de habilitar la interrupción, `main()` ejecuta la calibración de *offset* y la autocalibración, y vacía la FIFO.

Tabla 6: Configuración del LPADC0.

Parámetro	Valor
Reloj	FRO_HF, 48 MHz (asíncrono)
Resolución	Estándar: 12 bit single-ended
Canal	A0, lado A (kLPADC_SampleChannelSingleEndSideA)
Tiempo de muestreo	3 ciclos de ADCK (kLPADC_SampleTimeADCK3)
Promediado	Ninguno (1 conversión por disparo)
Referencia	kLPADC_ReferenceVoltageAlt1
Trigger	Trigger 0 → comando 1, por hardware
FIFO	FIFO0, marca de agua 0
Interrupción	Fin de trigger 0 (kLPADC_Trigger0CompletionInterruptEnable)
Calibración	<i>Offset</i> + autocalibración en <code>main()</code>

El VREF0 se enciende en modo *bandgap* sólo para dar la corriente de polarización que necesita el LPADC.

**Figura 5:** Configuración del LPADC0 mediante Config Tools.

2.5.3 Conversor D/A (DAC0)

Se usa el LPDAC0 de 12 bit sin FIFO: cada llamada a `DAC_SetData()` actualiza la salida directamente. La referencia es VREFH (kDAC_ReferenceVoltageSourceAlt3) y la salida pasa por el amplificador *buffer* interno. El módulo se enciende desde el SPC antes de inicializarlo.

Tabla 7: Configuración del DAC0.

Parámetro	Valor
Resolución	12 bit, dato justificado a derecha (0 a 4095)
FIFO	Deshabilitada (escritura directa)
Referencia de tensión	VREFH (kDAC_ReferenceVoltageSourceAlt3)
Referencia de corriente	PTAT interna
Buffer de salida	Habilitado
Salida	DAC0_OUT, P4_2

Low Power Digital-to-Analog Converter (LPDAC) [Peripheral drivers (Device specific)]

Name

Peripheral

▼ General configuration

Watermark value	0
FIFO trigger mode	Hardware
FIFO workmode	Disabled
Reference voltage source	VREFH_INT/VDD_ANA
Reference current source	PTAT
Opamp as buffer	☒
Periodic trigger number	0
Periodic trigger width	0
Synchronization time	1
Lower Low PowerMode	☐

Enable DMA for LPDAC ☐

FIFO DMA

Enable LPDAC ☒

Set LPDAC level after initialization ☐

LPDAC level

▼ Interrupt configuration

Interrupt sources	☐ FIFO full	☐ FIFO empty
	☐ FIFO watermark	☐ Swing back one cycle complete
	☐ FIFO overflow	☐ FIFO underflow
	☐ Period trigger complete	

Enable interrupt vector ☐

Figura 6: Config Tools — DAC0. Configuración del DAC de 12 bits.

2.5.4 Teclas y LED (GPIO)

Los tres segmentos del LED RGB se configuran como salidas GPIO; son activos en bajo, así que escribir 0 enciende el color. Las teclas se configuran como entradas con interrupción por flanco descendente (se presionan a masa). El GPIO0 tiene dos líneas de interrupción: SW3 (P0_6) usa la línea 0 (GPIO00_IRQn) y SW2 (P0_23) se asigna a la línea 1 (GPIO01_IRQn) mediante el campo IRQS del registro ICR[23]. Así cada tecla tiene su propia rutina de atención.

General Purpose Input/Output (GPIO) *[Peripheral drivers (Device specific)]*

Name: GPIO0

Peripheral: GPIO0

▼ **GPIO configuration**

Enable interrupt 0 ☒

Interrupt: GPIO00_IRQn

Interrupt request: Enabled in initialization

Enable priority initialization: ☐

Priority: 0

Enable custom handler name: ☐

Interrupt handler name: GPIO0_INT_0_IRQHANDLER

Handler template: [Copy to clipboard](#)

Interrupt controller reference: [NVIC](#)

Enable interrupt 1 ☒

Interrupt: GPIO01_IRQn

Interrupt request: Enabled in initialization

Enable priority initialization: ☐

Priority: 1

Enable custom handler name: ☐

Interrupt handler name: GPIO0_INT_1_IRQHANDLER

Handler template: [Copy to clipboard](#)

Interrupt controller reference: [NVIC](#)

Figura 7: Config Tools — GPIO0. Interrupciones de las teclas en las líneas 0 y 1 del GPIO0.

2.5.5 Prioridades de interrupción

Tabla 8: Interrupciones del sistema. No se modificó la prioridad por defecto, así que las tres tienen prioridad 0 (en el Cortex-M33 un número menor es mayor prioridad).

Fuente	Handler	Prioridad
LPADC0 (fin de conversión)	ADC0_IRQHandler	0
Tecla SW3 (velocidad)	GPIO00_IRQHandler	0
Tecla SW2 (Run/Stop)	GPIO01_IRQHandler	0

Con la misma prioridad ninguna interrupción desaloja a otra: si se presiona una tecla justo cuando termina una conversión, la segunda rutina espera a que termine la primera. Como ambas son cortas frente al período de muestreo más chico ($20,8\mu\text{s}$, unos 3100 ciclos del núcleo), la demora no provoca pérdida de muestras. Además el disparo del ADC no depende del CPU, así que esa espera no altera el instante de muestreo.

3 Diseño de la aplicación

3.1 Arquitectura general

La figura 8 muestra las partes de la aplicación. El camino de la señal (fila superior) funciona por completo a partir de hardware e interrupciones: el CTIMER0 dispara cada conversión a través del INPUTMUX, el LPADC0 deja el resultado en su FIFO y la rutina de interrupción lo guarda en el buffer circular y lo escribe en el DAC0. Las teclas sólo modifican el estado (índice de velocidad y modo Run/Stop): la de velocidad reprograma el match del CTIMER0 y el color del LED, y la de Run/Stop cambia lo que hace la rutina del ADC con cada muestra. El lazo principal queda vacío.

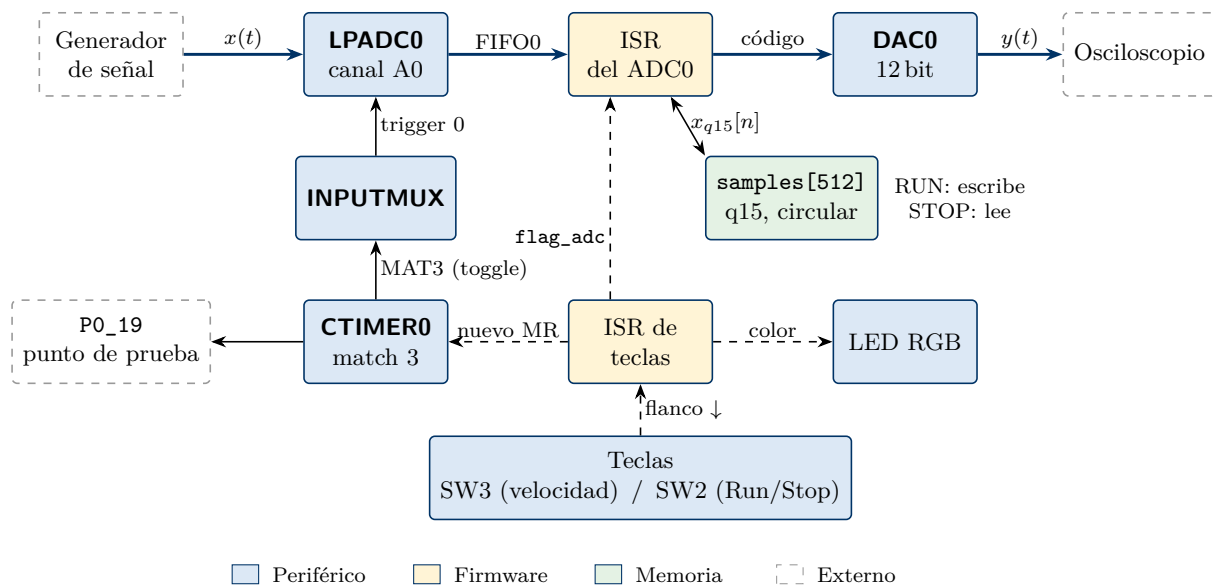


Figura 8: Diagrama de bloques de la aplicación. Trazo continuo azul: señal; negro: datos y disparo; punteado: control.

3.2 Generación de las velocidades de muestreo

Con el timer contando a f_{timer} , reiniciándose en el match y con la salida en toggle, un período completo de la señal de disparo abarca dos matches, es decir $2(\text{MR} + 1)$ ciclos del timer:

$$f_s = \frac{f_{\text{timer}}}{2(\text{MR} + 1)}. \quad (2)$$

En el firmware los valores de MR están en el arreglo `sample_vel` (constantes `VEL_0` a `VEL_4`) y se eligió para cada caso el entero más cercano a $f_{\text{timer}}/(2f_s) - 1$. La tabla 9 aplica la ecuación (2) con $f_{\text{timer}} = 150 \text{ MHz}$ a esos valores. El índice de estado coincide con la posición en el arreglo.

Tabla 9: Velocidades de muestreo, valor de match programado y color del LED.

Estado	f_s nominal	MR	f_s real	Error	LED
E0	8 000 Hz	9 374	8 000,00 Hz	0,000 %	● Verde
E1	16 000 Hz	4 687	15 998,29 Hz	−0,011 %	● Amarillo
E2	22 000 Hz	3 408	22 000,59 Hz	0,003 %	● Azul
E3	44 000 Hz	1 704	43 988,27 Hz	−0,027 %	● Rojo
E4	48 000 Hz	1 562	47 984,64 Hz	−0,032 %	○ Blanco

La presencia de $MR + 1$ en el denominador hace que los valores enteros de MR produzcan pequeños errores respecto de la velocidad nominal. Con los valores implementados, el error de cuantización es menor a 0,04% en todas las velocidades, por lo que resulta despreciable para esta aplicación.

Interpretación de 22K/S y 44K/S

Se tomaron literalmente como 22 kS/s y 44 kS/s. Si la cátedra se refiere a las frecuencias estándar de audio (22,05 kHz y 44,1 kHz), basta cambiar `VEL_2` y `VEL_3` en `source/TP1.c` por 3400 y 1700, y actualizar la tabla en `main.tex`.

3.3 Máquina de estados

El comportamiento de las teclas se modela con dos variables de estado independientes (figura 9):

- **state_counter**: índice de velocidad. Avanza en forma circular con SW3; al llegar a E4 vuelve a E0. Cada cambio reprograma el match del CTIMER0 y el color del LED.
- **flag_adc**: modo Run/Stop. Alterna con SW2. En RUN cada muestra se guarda en el buffer y se envía al DAC. En STOP el buffer queda congelado con las últimas 512 muestras y el DAC reproduce su contenido en forma cíclica, a la misma velocidad de muestreo.

Las dos teclas actúan en cualquier estado: se puede cambiar la velocidad estando en STOP (cambia la velocidad de reproducción). El LED sólo indica la velocidad, no el modo Run/Stop. Al reiniciar la placa el sistema arranca en E0 y en RUN.

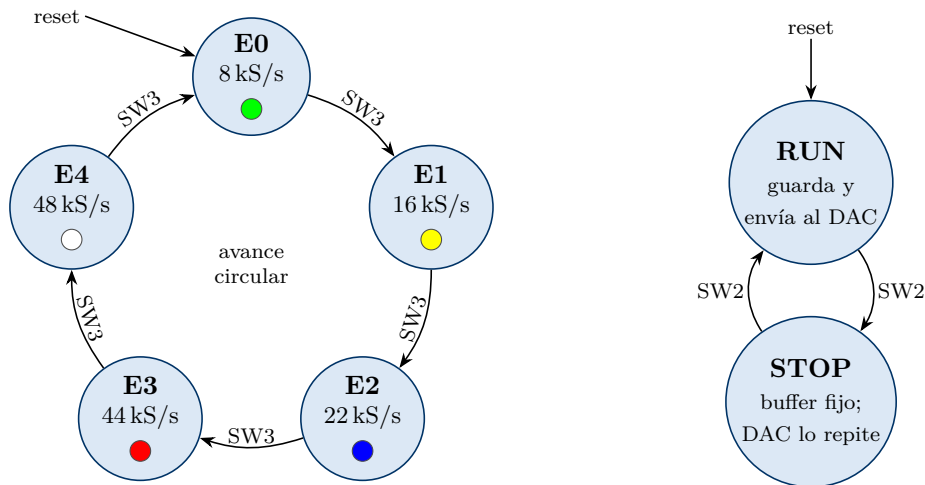


Figura 9: Izquierda: selección circular de la velocidad de muestreo con SW3 y color del LED en cada estado. Derecha: modo de adquisición con SW2.

3.4 Formato q15 y conversiones

Un número q15 es un entero de 16 bits con signo que representa el valor

$$v_{q15} = \frac{x_{q15}}{2^{15}} \in [-1, 1 - 2^{-15}]. \quad (3)$$

En resolución estándar el LPADC entrega el resultado de 12 bit alineado a la izquierda (bits 14:3 de la palabra de la FIFO). El firmware lo recupera con un corrimiento de 3 lugares:

$$v = (x_{ADC} \gg 3) \& 0x0FFF, \quad v \in [0, 4095]. \quad (4)$$

Como la entrada es single-ended, v es unipolar. Para que el buffer sea un q15 válido para CMSIS-DSP (centrado en cero y a escala completa) se resta la mitad de escala y se multiplica por 16:

$$x_{q15} = 16(v - 2048) \in [-32768, 32752]. \quad (5)$$

Así 0 V corresponde a $-1,0$, 1,65 V a 0 y 3,3 V a casi $+1$ (constantes ADC_SHIFT y ADC_MID en source/TP1.c).

En RUN el DAC recibe v directamente (monitoreo de la entrada). En STOP se invierte la conversión con saturación:

$$\text{código}_{\text{DAC}} = \text{sat}_{[0, 4095]} \left(\frac{x_{q15}}{16} + 2048 \right). \quad (6)$$

La conversión de ida y vuelta no pierde información (12 bits dentro de 16); la saturación sólo actúa si el buffer se procesa antes de reproducirlo.

3.5 Rutinas de interrupción

Toda la aplicación corre en tres rutinas de interrupción (figura 10). La del ADC se ejecuta una vez por muestra; las de las teclas, una vez por pulsación.

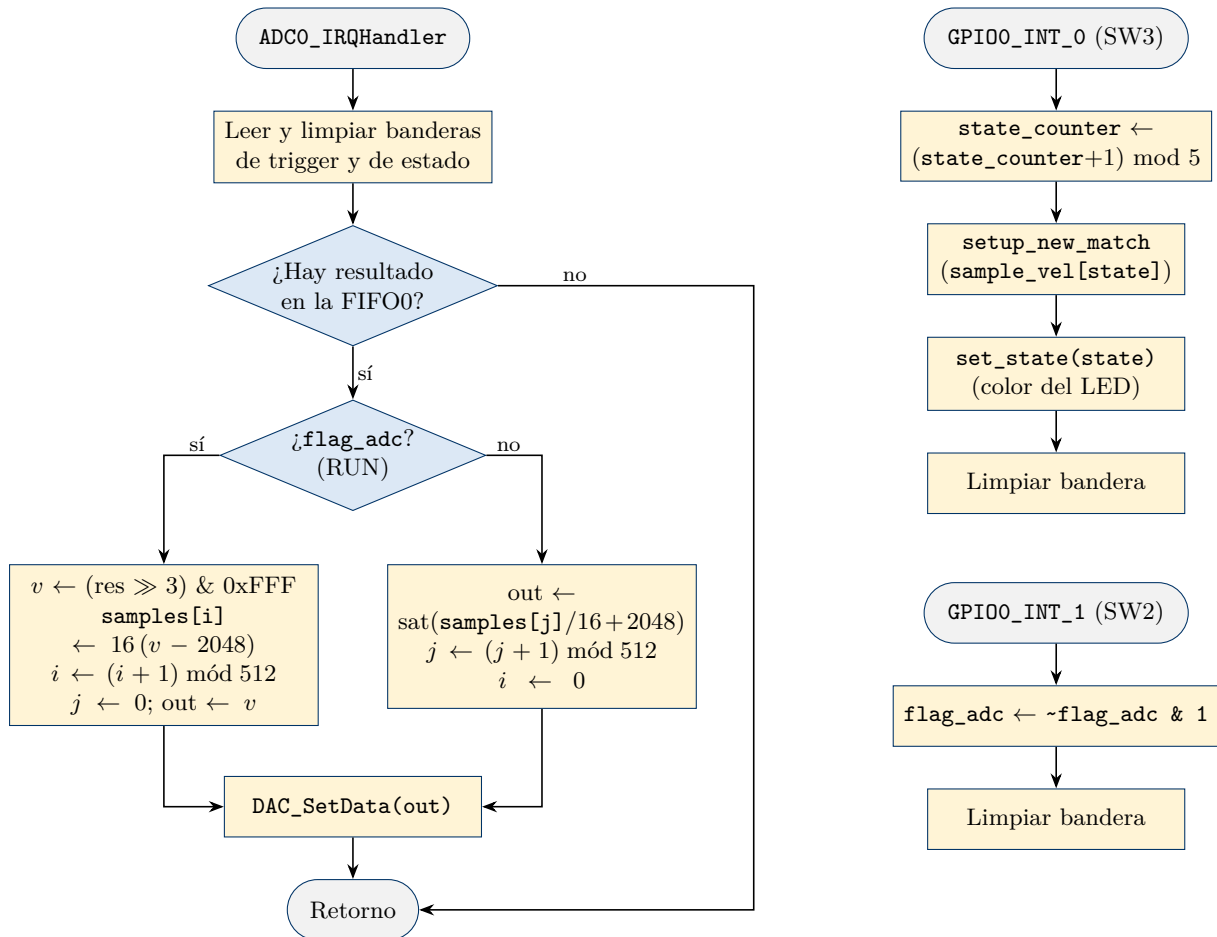


Figura 10: Diagramas de flujo de las rutinas de interrupción. Izquierda: fin de conversión del ADC. Derecha: teclas de velocidad (SW3) y de Run/Stop (SW2).

4 Implementación

Toda la lógica propia está en `source/TP1.c`; el resto del proyecto es código generado por Config Tools (sección 2) y drivers del SDK. El archivo completo está en el Anexo A.

4.1 Constantes y variables globales

Los valores de match de cada velocidad se agrupan en `sample_vel`, así que avanzar de estado es sólo avanzar un índice. Las variables que comparten las rutinas de interrupción se declaran `volatile`. El buffer `samples` es un arreglo de 512 elementos `q15_t`; su índice de escritura (`sample_count`) y el de lectura en STOP (`dac_count`) son variables estáticas de la rutina del ADC.

4.2 Programa principal

`main()` inicializa pines, relojes y periféricos con las funciones generadas por Config Tools, fija el color del estado inicial y calibra el ADC con la interrupción deshabilitada y el timer detenido, para que no lleguen disparos durante la calibración. Luego vacía la FIFO, habilita la interrupción, arranca el CTIMER0 y queda en un lazo vacío: a partir de ese momento todo ocurre en las interrupciones.

4.3 Cambio de velocidad

La función `setup_new_match()` detiene el CTIMER0, carga el nuevo valor de match con la misma configuración que generó Config Tools (reinicio en el match, salida en toggle), reinicia la cuenta y vuelve a arrancar el timer. Reiniciar la cuenta evita que, si el nuevo MR es menor que la cuenta actual, el contador tenga que dar la vuelta completa de 32 bits antes del próximo match.

`set_state()` traduce el índice de velocidad a los tres pines del LED. Como el LED es activo en bajo, un 0 enciende el segmento.

4.4 Rutinas de interrupción

La rutina de SW3 avanza el índice en forma circular (módulo del tamaño del arreglo), reprograma el timer y actualiza el LED. La de SW2 alterna `flag_adc` entre 0 y 1.

La rutina del ADC limpia las banderas, lee el resultado de la FIFO0 y, según el modo, guarda la muestra y la envía al DAC (RUN) o envía al DAC la muestra guardada siguiente (STOP). En ambos casos el índice avanza módulo 512, que es lo que hace circular al buffer.

5 Verificación

5.1 Procedimiento

1. Conectar el generador a J4[2] (ADC0_A0) con una senoidal de 2 kHz, amplitud 3 V y un *offset* que la mantenga entre 0 y 3,3 V.
2. Canal 1 del osciloscopio en la entrada, canal 2 en J1[4] (DAC0_OUT).
3. Recorrer las cinco velocidades con SW3, verificar el color del LED y medir en J2[13] (P0_19) la frecuencia de la señal de disparo, que es igual a f_s .
4. Presionar SW2 y verificar que la salida pasa a repetir el contenido del buffer aunque cambie la entrada; volver a presionar para reanudar.
5. Detener el programa con el depurador y observar el arreglo `samples` para comprobar que contiene 512 muestras de la señal.

5.2 Frecuencia de muestreo medida

Tabla 10: Frecuencia de la señal de disparo medida en P0_19.

Estado	f_s esperada [Hz]	f_s medida [Hz]	LED observado
E0	8000,00	8013	Verde
E1	15998,29	16020	Amarillo
E2	22000,59	22030	Azul
E3	43988,27	44050	Rojo
E4	47984,64	48060	Blanco

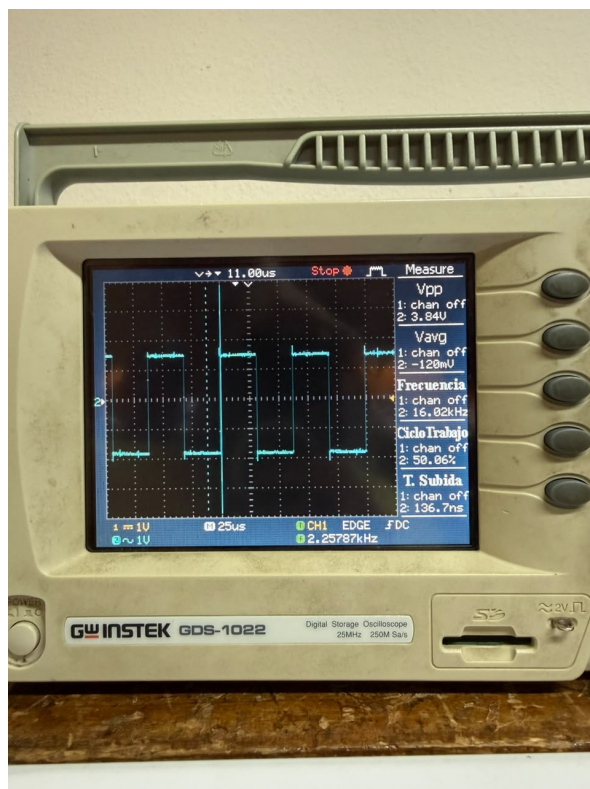


Figura 11: Osciloscopio. Señal de disparo del ADC en P0_19 (CT0_MAT3).

5.3 Señal de entrada y salida

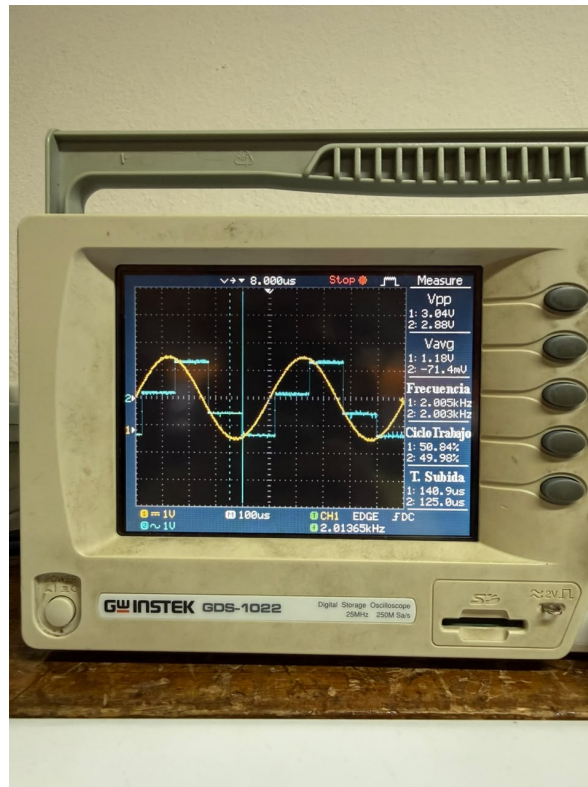


Figura 12: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) a 8 kS/s.

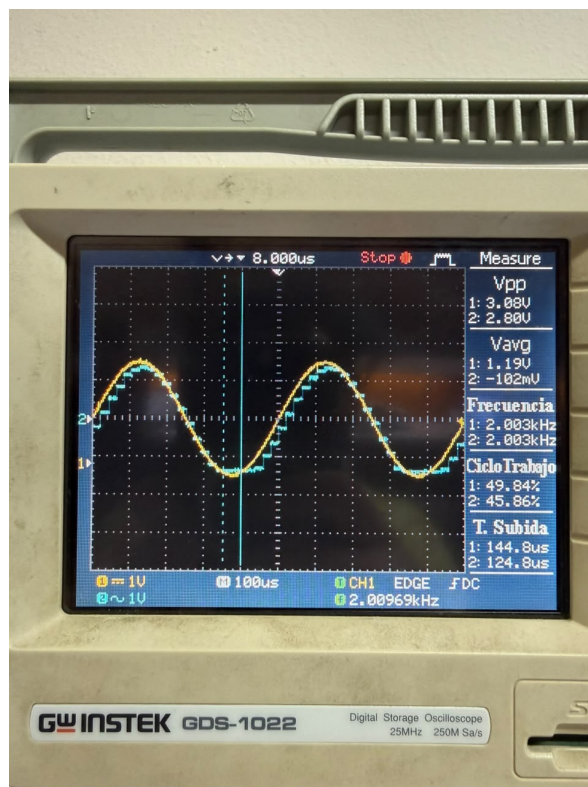


Figura 13: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) a 48 kS/s.

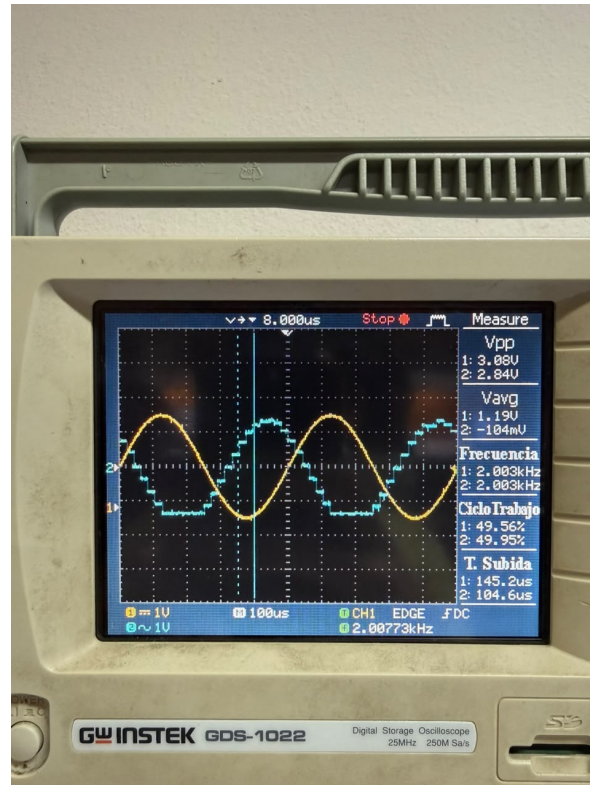


Figura 14: Osciloscopio. Entrada (canal 1) y salida del DAC (canal 2) con la adquisición detenida mediante SW2.

5.4 Cumplimiento de los requerimientos

Tabla 11: Verificación de los requerimientos de la consigna.

Req.	Cómo se implementa	Evidencia	Resultado
R1	CTIMER0 + INPUTMUX disparan el LPADC0 a la f_s elegida	Tabla 10	✓
R2	SW3 avanza <code>state_counter</code> módulo 5	Demostración	✓
R3	<code>set_state()</code> : un color por velocidad	Tabla 9	✓
R4	SW2 alterna <code>flag_adc</code>	Figura 14	✓
R5	DAC_SetData() en cada interrupción del ADC	Figuras 12 y 13	✓

6 Conclusiones

Se implementó sobre la FRDM-MCXN947 una aplicación que digitaliza una señal con el LPADC0 a cinco velocidades seleccionables (8 kS/s, 16 kS/s, 22 kS/s, 44 kS/s y 48 kS/s), guarda las muestras en un buffer circular de 512 valores `q15_t` y las envía al DAC0 de 12 bit. Con SW3 se recorren las velocidades en forma circular, cada una indicada por un color del LED RGB, y con SW2 se alterna entre adquisición y detención (Run/Stop). Los cinco requerimientos **R1–R5** se cumplieron, como resume la tabla 11; la reproducción en STOP supone haber adquirido previamente al menos 512 muestras para que todo el buffer contenga datos válidos.

La decisión principal del diseño fue disparar el ADC por hardware: la salida del match 3 del CTIMER0 llega al LPADC0 a través del INPUTMUX, sin intervención del CPU. Así el período de muestreo depende sólo del reloj del timer y no de la latencia de las interrupciones. El procesador interviene únicamente para mover cada resultado al buffer y al DAC, y a 48 kS/s quedan unos 3100 ciclos del núcleo por muestra.

Las mediciones validan este diseño. La señal de disparo observada en P0_19 (figura 11) es una onda cuadrada de ciclo de trabajo cercano a 50 % (50,06 % en la captura), con una frecuencia de 16,02 kHz en el estado E1. Esto confirma el modo *toggle* del match y la relación $f_s = f_{\text{timer}}/(2(\text{MR} + 1))$ de la ecuación (2). En los cinco estados la frecuencia medida (tabla 10) supera a la esperada en alrededor de 0,13 % a 0,17 %, una desviación sistemática y menor a 0,2 %. Es despreciable para esta aplicación.

En las capturas de entrada y salida (figuras 12, 13 y 14) la señal reconstruida en P4_2 conserva la frecuencia de la senoidal de entrada, de aproximadamente 2 kHz, y tiene forma de escalera, propia de un retenedor de orden cero. El ancho de cada escalón disminuye al aumentar f_s : a 8 kS/s hay sólo cuatro muestras por período y la salida es muy escalonada, mientras que a 48 kS/s sigue con mucha mayor fidelidad a la entrada. Además, la salida presenta un retardo respecto de la entrada, atribuible al retenedor de orden cero y a la latencia de la cadena de conversión.

Durante el desarrollo se resolvieron varios problemas técnicos. Primero, el resultado de 12 bit del LPADC viene alineado a la izquierda en la palabra de la FIFO, por lo que se aplicó un corrimiento de 3 lugares y se escaló a `q15` centrado en cero para que el buffer sea compatible con CMSIS-DSP, con conversión inversa y saturación para el DAC. Segundo, el CTIMER0 se reinicia al reprogramar el match, para que un MR menor que la cuenta actual no obligue al contador a dar una vuelta completa de 32 bits antes del próximo disparo. Tercero, la calibración del ADC se hace con la interrupción deshabilitada y el timer detenido. Cuarto, cada tecla se asignó a una línea de interrupción propia del GPIO0 mediante el campo `IRQS`, lo que evita compartir rutina de atención. Por último, se evaluó la ambigüedad de las velocidades de 22 kS/s y 44 kS/s, que se adoptaron literalmente (con su alternativa documentada en la sección 3).

En conjunto, la solución es robusta porque desacopla el instante de muestreo del software: ninguna latencia de interrupción altera el período de muestreo, y la espera entre rutinas de igual prioridad queda muy por debajo del período mínimo de 20,8 μs . La principal limitación observada es la forma escalonada de la salida del DAC, que en una aplicación real requeriría un filtro de reconstrucción y, para señales cercanas a $f_s/2$, un filtro antialiasing previo al ADC.

A Código fuente

El proyecto completo de MCUXpresso (incluidos los archivos generados por Config Tools, los drivers del SDK y CMSIS-DSP) está en la carpeta TP1/ del repositorio <https://github.com/Mati-Costamagna/procesamiento-digital-de-senales>.